

CodeVue: A Comprehensive Platform for Coding Practice and Problem Solving

(Submitted for the Partial Fulfillment of the Requirements for the Degree of Bachelor of Technology in Computer Science and Engineering)

J.C. Bose University of
Science and Technology,
Faridabad

SUBMITTED BY :-
Mohd Ujaid (21020004069)



Under the Guidance of

Assistant Professor
Name
(Project Guide)

Dr. Shalendra Kumar
(Principal)

DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING

Satyug Darshan Institute of Engineering & Technology, Faridabad

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of the **CodeVue** project. This project has been an incredible learning experience, and its successful completion would not have been possible without the contributions of several individuals.

First and foremost, I would like to thank **Ms. Name**, my project supervisor, for her constant guidance, insightful feedback, and encouragement. Her expertise and support were essential in shaping this project and ensuring its success.

I am also deeply grateful to my **friends** for their continuous support, motivation, and encouragement. Their belief in me kept me going during challenging times and helped me stay focused on the project.

Additionally, I would like to acknowledge the invaluable resources provided by the **open-source community**, which played a key role in helping me implement various technologies used in the project.

Finally, I extend my thanks to all my **teachers** for their guidance and the knowledge they imparted, which helped me complete the project with confidence.

This project is the result of collective effort, and I am truly thankful to everyone who contributed in any way.

Table of Content

S. No.	Content	Page No.
1	Abstraction	4
2	Introduction	5-6
3	REQUIREMENT ANALYSIS	6-9
4	DESIGN	10-12
5	Code Implementation	12-15
6	DATABASE AND DATA DICTIONARY	16-19
7	INFORMATION ABOUT TESTING STRATEGY	19-21
8	Project Snapshots of GUI	21-27
9	Conclusion	27-28
10	Future Scope	28-29
11	References	29-30

Abstract

CodeVue is an interactive, web-based platform designed to help students, developers, and aspiring programmers enhance their coding skills through real-time problem solving and coding challenges. Inspired by popular platforms like Leetcode, CodeVue allows users to practice coding in multiple programming languages, including JavaScript, Python, and Java. The platform offers an intuitive user interface, a dynamic code editor powered by Monaco Editor, and instant feedback on code execution.

The primary objective of CodeVue is to provide an efficient and engaging learning environment for both beginners and experienced programmers. By offering coding problems categorized by difficulty (easy, medium, and hard), users can track their progress, solve problems at their own pace, and prepare for technical interviews. The platform features multi-language support, allowing users to write, test, and submit solutions in different programming languages, making it accessible to a wider range of users.

CodeVue integrates with the **Judge0 API**, which allows users to execute code in real-time and receive immediate feedback, including execution results, runtime, memory usage, and error messages. Additionally, the platform supports secure user authentication, enabling users to track their submissions, view their problem-solving history, and manage their progress through a personalized dashboard.

Admin features enable the creation, management, and deletion of coding problems, ensuring a continuous stream of fresh content. CodeVue also includes a playlist feature that allows users to create custom collections of problems, further enhancing the learning experience by enabling targeted practice on specific topics or areas of weakness.

Built using modern web technologies such as **React.js**, **Node.js**, **Express.js**, **Prisma**, **PostgreSQL**, and **Tailwind CSS**, CodeVue is optimized for performance, scalability, and responsiveness across all devices. This project aims to revolutionize the way developers practice coding, offering a comprehensive solution for coding challenges, problem-solving, and interview preparation in a collaborative, secure, and engaging environment.

Introduction

In the digital age, programming proficiency is no longer limited to software engineers alone—it is a fundamental skill across diverse industries, including data science, cybersecurity, artificial intelligence, finance, and even the arts. However, one of the biggest challenges in learning programming is the lack of platforms that provide structured, practical, and interactive environments for coding practice. To bridge this gap, **CodeVue** has been developed as a comprehensive, web-based platform that enables users to improve their coding skills through real-time problem solving and practice in multiple programming languages.

CodeVue is inspired by widely used platforms such as **LeetCode**, **HackerRank**, and **Codeforces**, but is uniquely tailored to meet the academic and practical learning needs of students and developers. It serves as a learning tool as well as a performance evaluator, empowering users to prepare for competitive programming, technical interviews, and academic assessments. The system is particularly useful for Computer Science and Engineering students who wish to supplement their curriculum with hands-on experience in algorithms, data structures, and language-specific syntax.

The platform allows users to solve problems categorized by **difficulty level**—Easy, Medium, and Hard—and supports multiple languages including **JavaScript**, **Python**, and **Java**. The frontend is developed using **React.js** with **Tailwind CSS** for modern, responsive design, while the backend is powered by **Node.js** and **Express.js**. A **PostgreSQL** database with **Prisma ORM** handles structured data management, and **Judge0 API** is integrated for real-time code compilation and execution.

CodeVue offers the following major features:

- **Monaco-based code editor** for real-time syntax highlighting and editing
- **Dynamic problem descriptions** with examples, constraints, and hints
- **Instant execution feedback**, including runtime, memory usage, and error messages
- **User authentication and authorization** using JWT (JSON Web Tokens)

-
- **Admin dashboard** for creating and managing coding problems
 - **Submission tracking and history** for monitoring individual performance
 - **Playlists**, enabling users to create customized collections of problems for focused learning

Additionally, CodeVue supports secure user registration, login/logout functionality, and personal dashboards for submission tracking. The platform is built with scalability and performance in mind, ensuring an optimal user experience across desktop and mobile devices. To ensure a secure and reliable authentication mechanism, JSON Web Tokens (JWT) are employed, along with token blacklisting functionality to prevent unauthorized access.

The development of CodeVue is a practical demonstration of full-stack application design and deployment, combining frontend engineering, backend development, RESTful API design, database schema modeling, cloud-based testing (using tools like Postman), and integration of third-party services (Judge0). This project not only showcases technical proficiency but also emphasizes problem-solving, system design, and user-centered thinking.

REQUIREMENT ANALYSIS

Introduction

The Software Requirements Specification (SRS) for **CodeVue** outlines the functional and non functional requirements of the system. This section defines what the system will do, the features it must support, and the conditions it must satisfy. The aim is to provide a clear understanding of the system's capabilities, constraints, and overall objectives.

Purpose

The purpose of CodeVue is to offer an online coding platform where users can solve programming problems in real-time, track their progress, and receive immediate feedback.

The platform is designed to aid students and developers in enhancing their coding skills across different difficulty levels and programming languages.

Scope of the Project

CodeVue is a full-stack web application that enables:

- Solving algorithmic problems across various difficulty levels.
- Writing, executing, and testing code in multiple languages (JavaScript, Python, Java).
- Real-time feedback on runtime, memory usage, and execution status.
- User authentication and profile management.
- Admin functionality for adding, updating, and managing problems.
- Creation and management of playlists for organized learning.

Functional Requirements

These are the core features the system must provide:

User Management

- **User Registration and Login** using email, password, and name.
- **Secure Authentication** via JWT.
- **Logout Functionality** and token blacklisting.

Problem Management

- Admin can **add, update, and delete problems**.
- Each problem includes title, description, constraints, examples, code snippets, and test cases.

Code Execution

- Users can write and **execute code in real time** using Monaco Editor.
- System uses **Judge0 API** to compile and run code.
- Display of **runtime, memory, status, stdout, and stderr**.

Submission Management

-
- Store each code submission with associated language, status, and time.
 - Track history of all submissions.

Playlist Management

- Users can create playlists of problems.
- Add/remove problems from playlists.
- View playlist details.

Role Management

- Support for two roles: **Admin** and **User**.
- Admins have full access to manage problems and content.

Non-Functional Requirements

These define the performance and quality standards the system should meet:

Performance Requirements

- Code execution should return results within a few seconds.
- Support simultaneous execution requests from multiple users.

Security Requirements

- Passwords stored using strong hashing algorithms.
- JWT-based secure session handling.
- Role-based access control for APIs.

Usability

- Intuitive and user-friendly interface.
- Fully responsive design across devices (mobile, tablet, desktop).

Scalability

-
- Designed to support an increasing number of users and code submissions.

Reliability and Availability

- System must be available with minimal downtime.
- Handle invalid code and API failures gracefully.

Software and Hardware Requirements

Component	Minimum Specification
• Frontend	React.js, Tailwind CSS
• Backend	Node.js, Express.js
• Database	PostgreSQL, Prisma ORM
• Code Execution API	Judge0
• Authentication	JWT
• Testing Tool	Postman
• Version Control	Git, GitHub
• Browser	Chrome, Firefox, or any modern browser
• Hardware	4GB RAM, Dual Core CPU, Internet Access

Constraints

- Execution speed depends on the response time of the third-party Judge0 API.
- Free-tier Judge0 instance may impose rate limits.
- Requires internet connectivity for all core functionalities.

Assumptions

- Users have a basic understanding of programming languages.
- The backend server has access to Judge0 and a properly configured database.
- Admins understand how to manage coding problems and submissions.

DESIGN

This chapter explains the design of **CodeVue**, a coding practice platform inspired by LeetCode. The system is structured to be modular, secure, and scalable. It uses a client-server architecture, with well-defined components for frontend, backend, database, and external code execution services.

High-Level Design (HLD)

1.) System Architecture

CodeVue follows a **three-tier architecture**:

- **Frontend (Client):** Built with React.js and Tailwind CSS, it offers a responsive user interface with the Monaco Editor for real-time code writing.
- **Backend (Server):** Built using Node.js and Express.js, it handles API requests, authentication, problem management, and submission logic.
- **Database:** PostgreSQL is used to store user data, problems, submissions, and playlists. Prisma ORM manages database interaction.
- **External API:** Judge0 API is integrated to compile and execute user-submitted code in real time.

2.) Main System Components

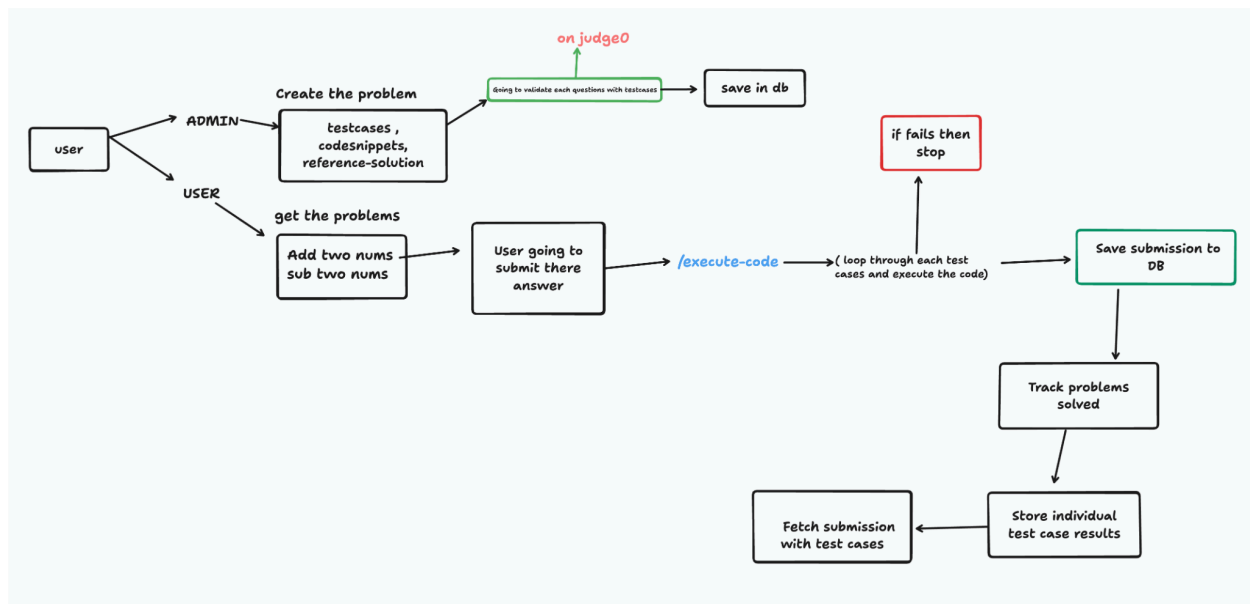
- **Authentication Module:** Handles registration, login, logout, and secure access using JWT.
- **Problem Management Module:** Allows users (especially admins) to add, edit, or delete coding problems.
- **Code Execution Module:** Sends user code to Judge0 API and returns output or errors.

- **Submission Module:** Stores and tracks each user's submissions with status (Accepted, Wrong Answer, etc.).
- **Playlist Module:** Users can create playlists and add problems for practice.

3.) Data Flow Design

Level 0 – Context Diagram

The system interacts with:



- **User/Admin** – Inputs: problems, code; Receives: results, feedback.
- **Judge0 API** – Executes code and returns results.

Level 1 – Process Breakdown

Module	Inputs	Outputs
Authentication	Email, password	JWT token, user profile
Problem Management	Problem details	Problem added/updated/deleted
Code Execution	Source code, language, input	Output, runtime, memory, status
Submission Tracking	User submissions	Submission history, status

4.) Database Design (Simplified)

Table Name	Key Fields	Purpose
User	id, name, email, password	Stores user details
Problem	id, title, difficulty, tags	Stores coding questions
Submission	id, userId, problemId, status	Records user attempts
Playlist	id, name, userId	User-defined problem sets
ProblemSolved	userId, problemId	Tracks solved problems
TokenBlacklist	token, expiresAt	Handles JWT invalidation

5.) Design Considerations

- **Security:** JWT-based access control with token blacklisting.
- **Scalability:** Easy to add more languages, problems, or features.
- **Responsiveness:** Tailwind CSS ensures UI works on all devices.
- **Reusability:** Clean separation of components and RESTful APIs.

Code Implementation

The project, **CodeVue**, was independently developed by the student Mohd Ujaied as a final year B.Tech submission. The student implemented all core modules of the platform, including user authentication, problem management, code execution using Judge0 API, submission tracking, and playlist features. The platform supports multiple programming languages and is built using modern web technologies such as React.js, Node.js, Express.js, and PostgreSQL. The project showcases the student's full-stack development skills, practical understanding of software architecture, and ability to build a complete, real-world web application from scratch.

Modules Implemented

User Authentication and Authorization Module

Purpose: To securely manage user registration, login, and access to protected routes.

Key Features:

- User registration and login using email and password.
- Passwords hashed securely using `bcrypt`.
- JWT (JSON Web Token) used for session management and access token generation.
- Role-based access control (Admin vs. User).
- Token Blacklisting to prevent unauthorized reuse of tokens.

Learning Outcome: Implemented secure authentication flows and middleware for protected routes, enhancing security understanding.

Problem Management Module

- **Purpose:** To allow admin users to create, update, retrieve, and delete coding problems from the platform.
- **Key Features:**
 - CRUD operations for problem data (title, description, examples, constraints, etc.).
 - Tagging and categorization of problems based on difficulty and topics.
 - Rich content support including hints and editorial solutions.
 - Admin-level restrictions for problem management.
- **Learning Outcome:** Gained experience with RESTful API design and backend validation techniques.

Code Execution and Submission Module

- **Purpose:** To allow users to write and submit code solutions, and receive automated execution results.
- **Key Features:**
 - Monaco Editor integration for an IDE-like experience.
 - Multi-language support: JavaScript, Python, Java.
 - Code submitted to Judge0 API for remote execution.
 - Captures outputs, error messages, execution time, and memory usage.
 - Submissions are saved with full metadata including problem ID and user ID.

-
- **Learning Outcome:** Learned how to integrate external APIs securely and handle asynchronous execution flows in a real-time environment.

Submission History and Status Tracking Module

- **Purpose:** To track, store, and display each user's code submission records.
- **Key Features:**
 - Displays all previous submissions by a user with details like runtime, memory, and status.
 - Users can see which problems they've attempted and whether their submissions were successful.
 - Stores detailed execution status such as Accepted, Compilation Error, Runtime Error, Wrong Answer, etc.
- **Learning Outcome:** Developed efficient querying and filtering techniques using Prisma ORM and PostgreSQL.

Playlist Management Module

- **Purpose:** To enable users to create and manage custom playlists of problems for targeted practice.
- **Key Features:**
 - Create, rename, or delete playlists.
 - Add or remove problems from playlists.
 - Ensure uniqueness of playlist names per user and prevent duplicate problems.
- **Learning Outcome:** Understood many-to-many relationships and how to model them effectively using relational databases.

Frontend Integration and Interface Development

- **Purpose:** To design a seamless and responsive user interface that supports all core features.
- **Key Features:**
 - Built using React.js and Tailwind CSS for fast and clean UI.
 - Integrated dynamic forms for problem creation and submission.

-
- Protected routes for user dashboard and admin controls using JWT tokens.
 - Real-time form validation and conditional rendering.
 - **Learning Outcome:** Gained expertise in component-based UI development and routing using React Router.

Tools and Technologies Used

- **Frontend:** React.js, Tailwind CSS, Monaco Editor
- **Backend:** Node.js, Express.js, JWT, Prisma ORM
- **Database:** PostgreSQL
- **Code Execution:** Judge0 API
- **API Testing:** Postman
- **Version Control:** Git & GitHub
- **Other Tools:** Prisma Studio, VS Code

Challenges Faced

- Managing asynchronous code execution from Judge0 and synchronizing the response.
- Ensuring secure authentication and preventing token reuse through blacklisting.
- Designing an intuitive, error-free form for adding complex problem data (JSON for test cases, code snippets).
- Handling role-based access control across different modules

Learning Outcome

Through the successful implementation of the above modules, the student acquired practical skills in:

- Full-stack web development using modern JavaScript frameworks.
- Building scalable REST APIs and integrating third-party services.
- Designing normalized relational databases and working with ORMs.
- Implementing secure authentication and handling user roles.
- Creating responsive and interactive frontend interfaces.

DATABASE AND DATA DICTIONARY

The backend of **CodeVue** is powered by **PostgreSQL**, a robust and scalable relational database system. It is used to store and manage all core entities such as users, coding problems, submissions, playlists, and authentication data. The database structure is defined and managed using **Prisma ORM**, which allows seamless type-safe interaction between the backend logic and the database.

The database schema is designed with normalization principles to ensure data integrity and efficient data retrieval. Relationships are established through foreign keys and Prisma's relation mappings.

Data Dictionary

Below is a summary of the key tables used in the system, along with their primary fields and descriptions.

User Table

Field	Type	Description
id	UUID	Primary key, uniquely identifies a user
name	String	Username (unique)
email	String	Email address (unique)
password	String	Hashed password
image	String	Optional profile image URL
role	Enum	User role: ADMIN or USER
createdAt	DateTime	Timestamp of creation
updatedAt	DateTime	Timestamp of last update

Problem Table

Field	Type	Description
-------	------	-------------

id	UUID	Primary key
title	String	Title of the problem
description	String	Full problem description
difficulty	Enum	Difficulty level: EASY, MEDIUM, HARD
tags	String[]	Tags related to the problem
examples	JSON	Language-specific input/output examples
constraints	String	Constraints related to the problem
hints	String	Optional hints or tips
editorial	String	Solution explanation
testCases	JSON	Input/output pairs for validation
codeSnippets	JSON	Starter code in supported languages
referenceSolutions	JSON	Model solutions in each language
userId	UUID	Foreign key – creator of the problem
createdAt	DateTime	Timestamp of creation
updatedAt	DateTime	Timestamp of last update

Submission Table

Field	Type	Description
id	UUID	Primary key
userId	UUID	Foreign key – user who submitted
problemId	UUID	Foreign key – associated problem
sourceCode	JSON	Code submitted by the user
language	String	Programming language used
stdin	String	Optional input provided by the user
stdout	String	Output generated
stderr	String	Runtime errors (if any)

compileOutput	String	Compilation errors (if any)
status	String	Submission status (e.g., Accepted, Wrong Answer)
memory	String	Memory used
time	String	Time taken to execute
createdAt	DateTime	Timestamp of submission

Playlist Table

Field	Type	Description
id	UUID	Primary key
name	String	Name of the playlist
description	String	Optional description of the playlist
userId	UUID	Foreign key – creator of the playlist
createdAt	DateTime	Timestamp of creation
updatedAt	DateTime	Timestamp of last update

ProblemInPlaylist Table

Field	Type	Description
id	UUID	Primary key
playlistId	UUID	Foreign key – associated playlist
problemId	UUID	Foreign key – problem included in playlist
createdAt	DateTime	Timestamp of when the entry was created

ProblemSolved Table

Field	Type	Description
id	UUID	Primary key
userId	UUID	Foreign key – user who solved the problem
problemId	UUID	Foreign key – problem that was solved

createdAt	DateTime	Timestamp of when the problem was solved
-----------	----------	--

TokenBlacklist Table

Field	Type	Description
id	UUID	Primary key
token	String	JWT token that has been blacklisted
expiresAt	DateTime	When the token should be considered invalid
createdAt	DateTime	Timestamp of entry creation

TestCaseResult Table

Field	Type	Description
id	UUID	Primary key
submissionId	UUID	Foreign key – linked to a submission
testCase	Int	Test case number
passed	Boolean	Whether the test case passed or failed
stdout	String	Output from execution
expected	String	Expected output
stderr	String	Any runtime errors
compileOutput	String	Any compile-time errors
status	String	Status of test case execution
memory	String	Memory used
time	String	Time taken to execute
createdAt	DateTime	Timestamp of result

INFORMATION ABOUT TESTING STRATEGY

Testing is a critical part of the software development lifecycle to ensure the reliability, performance, and correctness of the application. For **CodeVue**, a combination of **manual testing**, **automated testing**, and **API testing** strategies was used to validate functionality, ensure user experience, and verify backend integration with the Judge0 API and PostgreSQL database.

Types of Testing Used

1. Unit Testing

- **Objective:** Validate individual components and functions in isolation.
- **Scope:** Backend utility functions, authentication middleware, and database services.
- **Tools Used:** Jest (Node.js Testing Framework)

2. Integration Testing

- **Objective:** Ensure modules interact correctly — especially API endpoints with the database and Judge0.
- **Scope:** API routes for problem creation, user authentication, and code submissions.
- **Tools Used:** Postman, Supertest (for testing Express.js routes)

3. Functional Testing

- **Objective:** Test the features from an end-user perspective to ensure each function behaves as expected.
- **Scope:** Code editor interactions, code execution, registration, login, problem browsing.
- **Tools Used:** Manual browser testing, browser console logs.

4. API Testing

- **Objective:** Ensure RESTful APIs return correct status codes, response data, and handle errors gracefully.
- **Scope:** Authentication, problem CRUD operations, submission tracking, playlist management.

-
- **Tools Used:** Postman Collection (structured with pre-request scripts and environment variables)

5. UI Testing

- **Objective:** Confirm that the user interface works seamlessly and is responsive across different devices.
- **Scope:** Responsiveness of editor, problem list rendering, user dashboard.
- **Tools Used:** Manual browser testing across Chrome, Firefox, and mobile views using Chrome Developer Tools.

Judge0 API Testing

Special attention was given to testing the code execution flow:

- **Tested multiple languages:** JavaScript, Python, and Java.
- **Simulated different outputs:** Correct outputs, runtime errors, timeouts, and compilation errors.
- **Verified:** Real-time feedback, result handling, and display of execution data.

Security Testing

Security tests focused on:

- **JWT Authentication:** Token expiration, access control for protected routes.
- **User Roles:** Preventing unauthorized access to admin routes.
- **Token Blacklisting:** Ensuring blacklisted tokens are rejected after logout.

Bug Tracking and Issue Handling

- Bugs were tracked and documented using **GitHub Issues**.
- Fixes and improvements were version-controlled with descriptive commit messages.

Project Snapshots of GUI

Fig.1)Signup Page

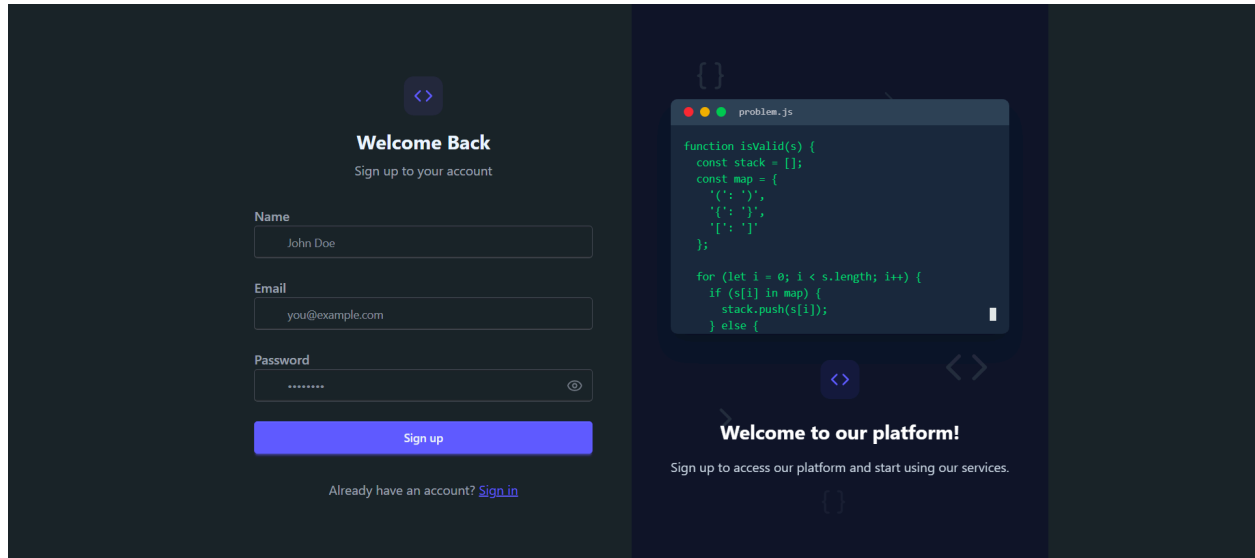


Fig2.) Sign In Page

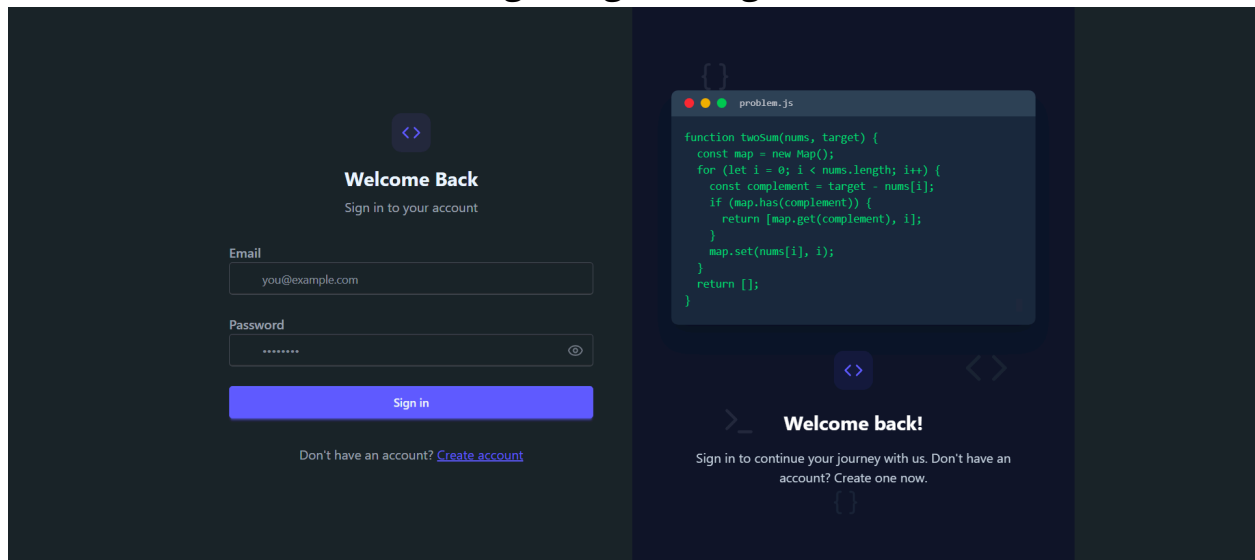


Fig.3) Home Page

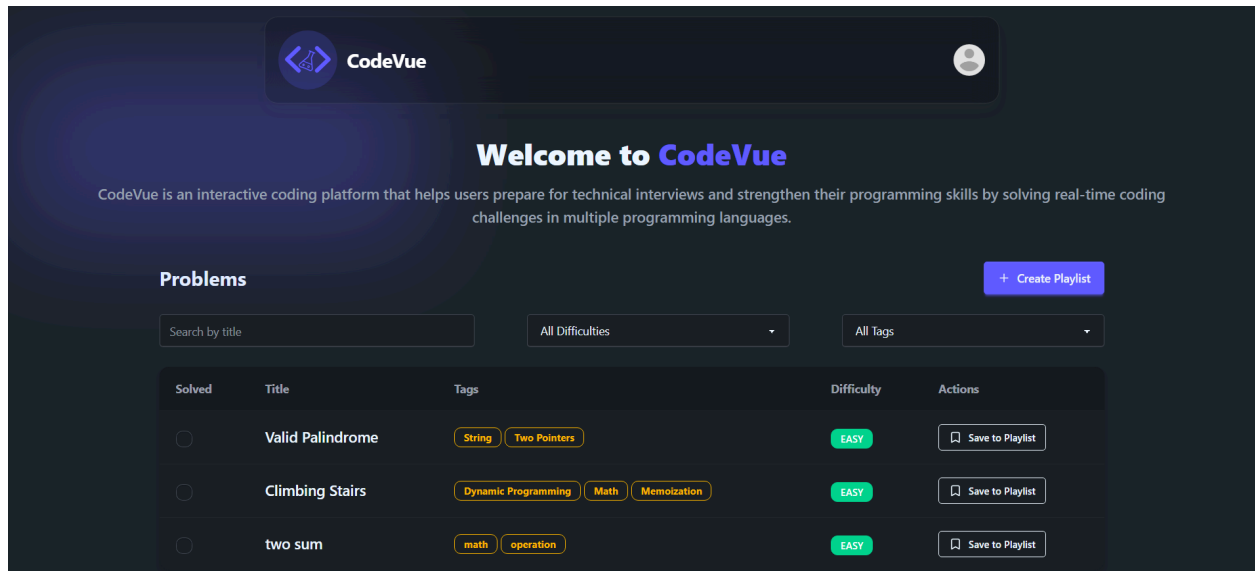
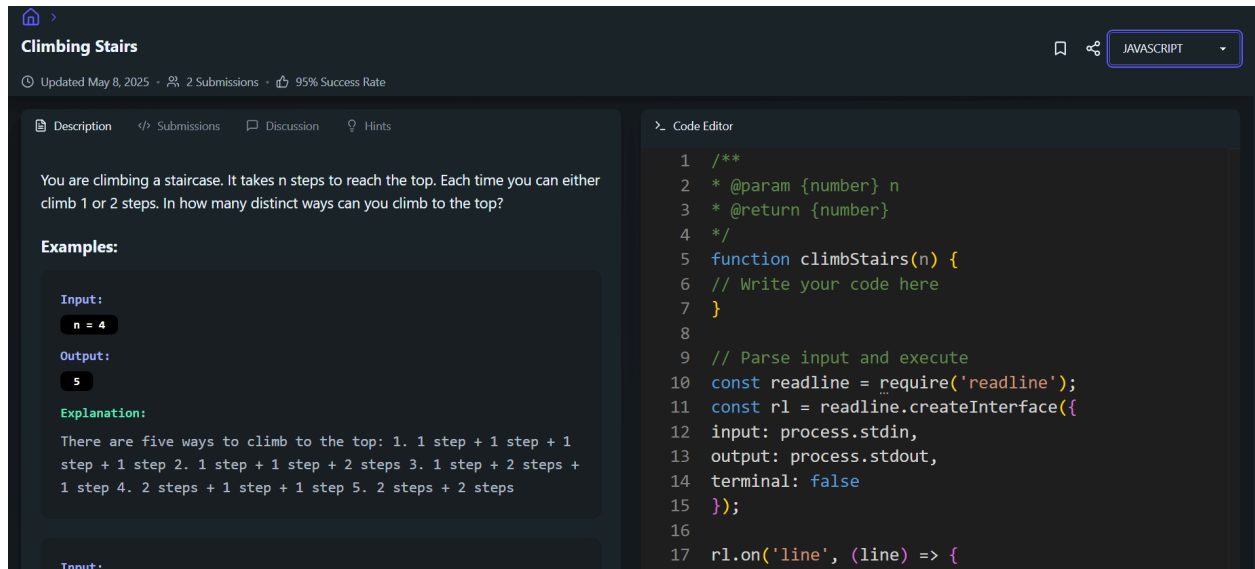


Fig.4) Problem Page



There are five ways to climb to the top: 1. 1 step + 1 step + 1 step + 1 step 2. 1 step + 1 step + 2 steps 3. 1 step + 2 steps + 1 step 4. 2 steps + 1 step + 1 step 5. 2 steps + 2 steps

Input:
n = 3

Output:
3

Explanation:
There are three ways to climb to the top: 1. 1 step + 1 step + 1 step 2. 1 step + 2 steps 3. 2 steps + 1 step

Input:
n = 2

Output:
2

Explanation:
There are two ways to climb to the top: 1. 1 step + 1 step 2. 2 steps

Constraints:
1 <= n <= 45

```

12 input: process.stdin,
13 output: process.stdout,
14 terminal: false
15 });
16
17 rl.on('line', (line) => {
18   const n = parseInt(line.trim());
19   const result = climbStairs(n);
20

```

Run Code Submit Solution

Test Cases

Input	Expected Output
2	2
3	3
4	5

Fig.5) Submissions detail

Climbing Stairs
Updated May 8, 2025 · 2 Submissions · 95% Success Rate

Description
Submissions
Discussion
Hints

Wrong Answer
JavaScript
0.019 s
7329 KB
5/8/2025

Accepted
JavaScript
0.018 s
7555 KB
5/8/2025

Code Editor

```

1 /**
2  * @param {number} n
3  * @return {number}
4  */
5 function climbStairs(n) {
6   // Write your code here
7 }
8
9 // Parse input and execute
10 const readline = require('readline');
11 const rl = readline.createInterface({
12   input: process.stdin,
13   output: process.stdout,
14   terminal: false
15 });
16
17 rl.on('line', (line) => {

```

Fig.6) Language and Discussion

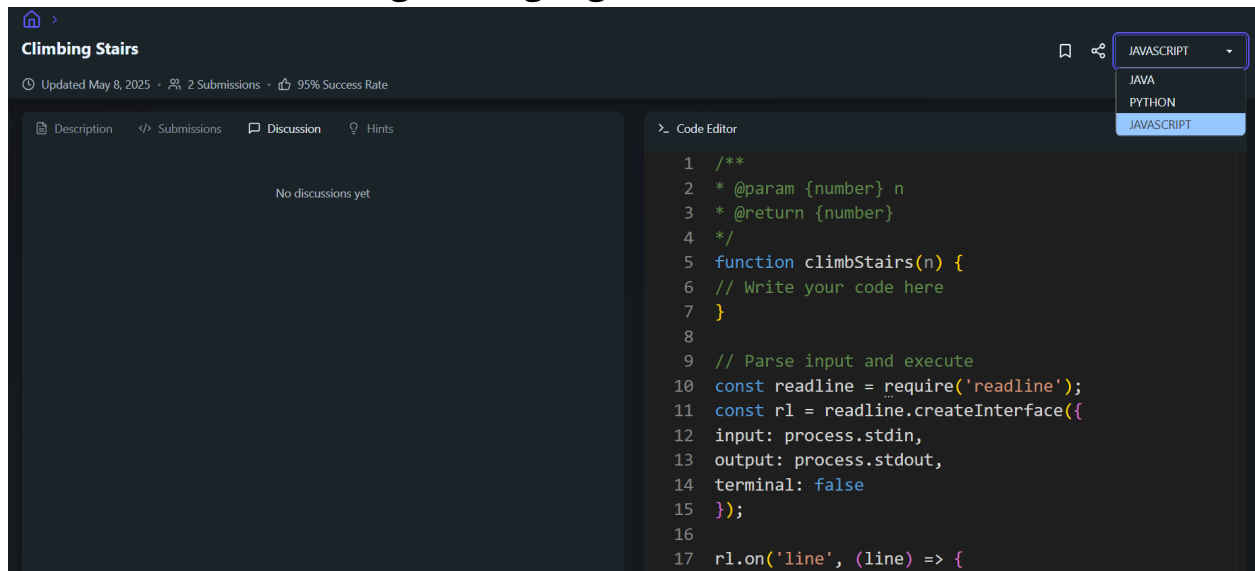


Fig.7) Profile Page

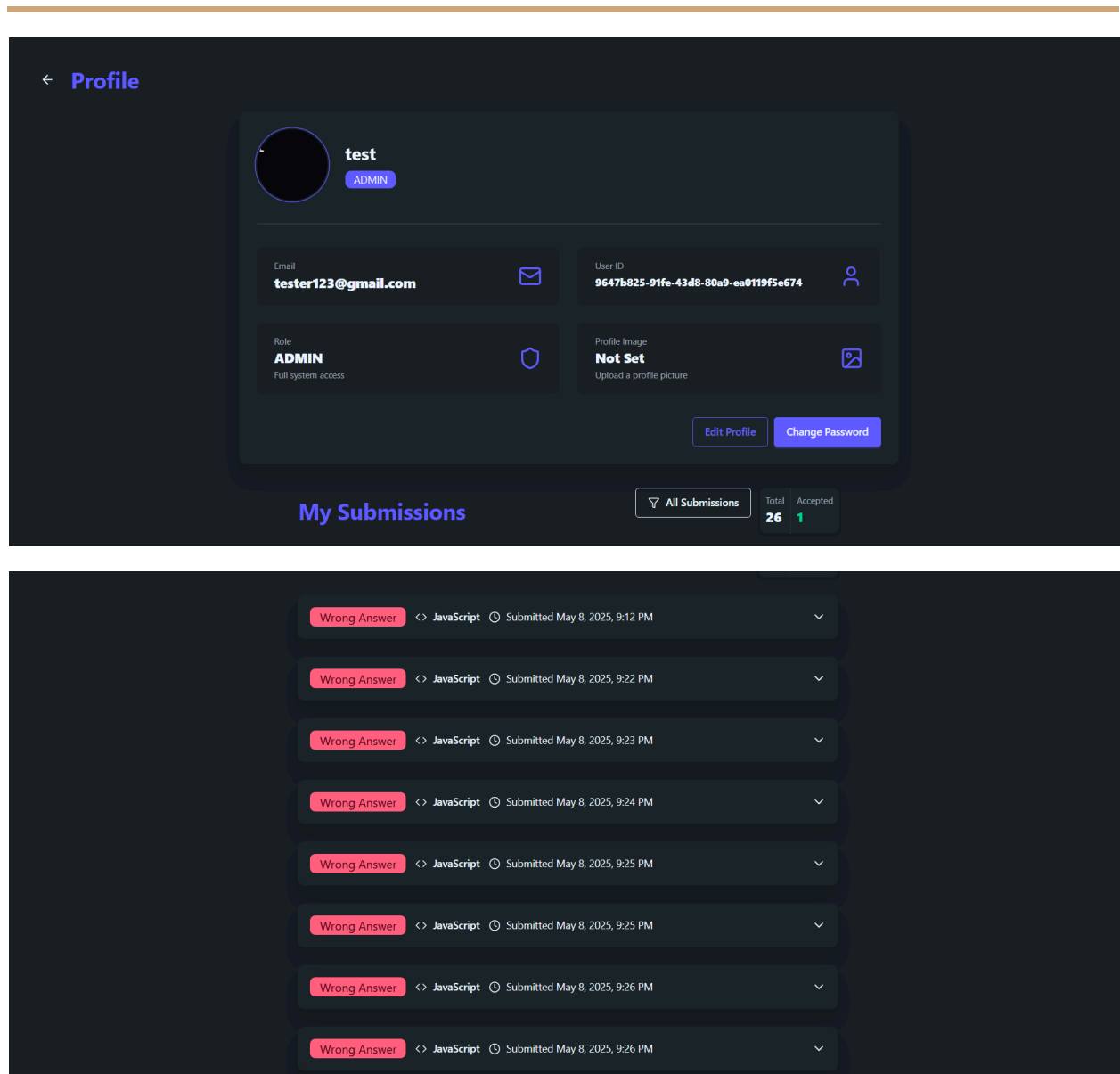


Fig.8) Submission Detail

The screenshot shows a code editor interface with a dark theme. At the top, a green status bar indicates the code is 'Accepted' and it's for 'JavaScript', submitted on May 8, 2025, at 8:52 PM. Below this is a section titled 'Solution Code' containing a JavaScript function 'climbStairs(n)' that calculates the number of ways to climb stairs. The code uses a loop to calculate Fibonacci-like values. Below the code editor are two panels: 'Input' and 'Output'. The 'Input' panel shows the numbers 2, 3, and 4. The 'Output' panel shows the result 235. At the bottom, there are statistics: 'Execution Time' is 0.017 s and 'Memory Used' is 7268 KB.

```
/**
 * @param {number} n
 * @return {number}
 */
function climbStairs(n) {
  // Write your code here
  if (n <= 1) return 1;

  let a = 1; // ways(0)
  let b = 1; // ways(1)

  for (let i = 2; i <= n; i++) {
    let temp = a + b;
    a = b;
    b = temp;
  }

  return b;
}

// Parse input and execute
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
  terminal: false
});

rl.on('line', (line) => {
  const n = parseInt(line.trim());
  const result = climbStairs(n);

  console.log(result);
  rl.close();
});
```

Input

2
3
4

Output

235

Execution Time
0.017 s

Memory Used
7268 KB

CONCLUSION

The **CodeVue** project successfully demonstrates the design and development of a comprehensive, user-friendly coding practice platform inspired by competitive programming websites like LeetCode. This system allows users to solve problems across multiple programming languages such as JavaScript, Python, and Java, with real-time code execution powered by the Judge0 API.

Throughout the development process, modern web technologies like **React.js**, **Tailwind CSS**, **Node.js**, **Express.js**, **PostgreSQL**, and **Prisma ORM** were effectively utilized to create a robust full-stack application. Features like dynamic problem descriptions, user

authentication with JWT, secure code submission, admin-controlled problem management, and Docker-based deployment highlight the project's technical depth and practical usability.

This project not only enhanced the student's understanding of full-stack development and system architecture, but also introduced real-world software engineering practices such as API design, database modeling, authentication, version control, and testing.

In conclusion, **CodeVue** stands as a scalable and extensible platform that meets its objectives of helping students and developers improve their coding skills in a practical, interactive, and structured environment. With further enhancements, such as real-time collaboration, leaderboard integration, and detailed analytics, this platform has the potential to evolve into a full-fledged educational tool for coding enthusiasts.

FUTURE SCOPE

While **CodeVue** successfully delivers a functional and interactive coding platform, there are several opportunities to enhance and expand its capabilities in future development phases:

1. **Real-Time Collaborative Coding**

Introduce pair programming or group problem-solving sessions where users can code together in real-time.

2. **Leaderboard and Gamification**

Add features like ranking systems, badges, streaks, and rewards to motivate consistent user engagement.

3. **Detailed Analytics and Performance Tracking**

Enable users to track progress over time, identify weak areas, and receive problem recommendations based on performance.

4. **Multi-Role User Access**

Add roles for mentors, reviewers, or instructors who can assign tasks, review code, and provide feedback.

5. **Mobile App Integration**

Develop a mobile version of CodeVue for on-the-go coding practice and accessibility.

6. **Support for More Languages and IDE Features**

Expand the number of supported programming languages and integrate features like code linting, autocompletion, and debugging.

7. **Discussion Forum and Community Features**

Implement a platform for users to discuss problems, share solutions, and interact with peers.

8. **Offline Mode and Problem Export**

Allow users to download problems and solve them offline, then sync submissions once online.

References

1. **Judge0 API Documentation** – Used for code execution and evaluation in multiple languages.

URL: <https://judge0.com>

2. **React.js Documentation** – For building the front-end user interface.

URL: <https://react.dev/learn>

3. **Tailwind CSS Documentation** – For designing responsive and modern UI.

URL: <https://tailwindcss.com/docs>

4. **Node.js Documentation** – Backend runtime environment for server logic.

URL: <https://nodejs.org/en/docs>

5. **Express.js Documentation** – Used to create RESTful APIs.

URL: <https://expressjs.com>

6. **Prisma ORM Documentation** – For database interaction with PostgreSQL.

URL: <https://www.prisma.io/docs>

7. **PostgreSQL Documentation** – For managing relational data.

URL: <https://www.postgresql.org/docs>

8. **JWT (JSON Web Tokens) Guide** – Used for secure authentication.

URL: <https://jwt.io/introduction>

-
9. **Docker Documentation** – For containerizing the application.
URL: <https://docs.docker.com>
 10. **Postman Learning Center** – For testing and managing API requests.
URL: <https://learning.postman.com>
 11. **Git & GitHub Guides** – For version control and collaboration.
URL: <https://docs.github.com/en>
 12. **Monaco Editor GitHub** – For integrating the interactive code editor.
URL: <https://github.com/microsoft/monaco-editor>
 13. **Visual Studio Code** – IDE used during project development.
<https://code.visualstudio.com>